Linear Eigenproblems in Photonic Crystals

Version: Python 3.12+ MATLAB R2023a+
Papers: DOI Paper 1 DOI Paper 2

Platform: GPU RTX 4090 24GB CUDA 12.4

Core Library 🚀: CuPy for accelerated linear algebra.

Overview

This repository contains implementations for two related papers:

Paper	Folder	Status
Paper 1	`paper_1_matlab/`, `paper_1_python/`	Historical archive
Paper 2	`paper_2/`	**Main implementation** — supersedes Paper 1

> **Note:** The `paper_2/` implementation is the recommended codebase. It reproduces all examples from Paper 1 with improved structure and time-efficient functions.

The structure of the repository is outlined as follows:

```
├── paper_1_matlab/          # Paper 1 MATLAB code (archive)
├── paper_1_python/          # Paper 1 Python code (archive)
├── paper_2/                 # Paper 2 – main implementation
|   ├── dielectric_examples/     # (folder) Storing dieletric indices
|   ├── output/                  # Results of bandgap
|   ├── _kernels.py              # CUDA kernels for Python
|   ├── dielectric.py            # I/O and computation of dielectric indices
|   ├── discretization.py        # Mimetic finite difference operators
|   ├── lobpcg.py                # LOBPCG eigensolver
|   ├── numerical_experiments.py # Experiment runners
|   ├── orthogonalization.py     # Orthogonalization utilities for LOBPCG
|   ├── pcfft.py                 # FFT-based matrix-free operations
|   └── run.sh                   # Shell for quick start
├── pic/                     # Visualization assets
└── README.md
```

##

Mathematical Theory

Computational framework for photonic bandgap analysis using finite difference (Yee's scheme) with kernel compensation. The mathematical model problem is given by:

$$\begin{cases} \nabla \times (\varepsilon^{-1} \nabla \times \boldsymbol{H}) = \omega^2 \boldsymbol{H}, & \text{in } \mathbb{R}^3, \\ \nabla \cdot \boldsymbol{H} = 0, & \text{in } \mathbb{R}^3, \end{cases}$$

where $\boldsymbol{H}$ is the magnetic field, ω is the frequency and $\varepsilon = \varepsilon(\omega, \boldsymbol{x})$ is the dielectric function. In this project, ε depends ONLY on the spatial position $\boldsymbol{x}$, resulting in linear eigenproblems. The case where ε is frequency-dependent (which results in nonlinear eigenproblems) is considered and solved in the repository https://github.com/Epsilon-79th/NHEP-nonlinear-Hermitian-eigenproblem-in-photonic-crystals. We summarize the

treatments towards the model problem as follows:

- **Bloch theorem** The magnetic field $\boldsymbol{H}$ satisfies the quasi-periodic boundary condition $\boldsymbol{H}(\boldsymbol{x}+\boldsymbol{a}_i) = e^{i\mathbf{k}\cdot\boldsymbol{a}_i}\boldsymbol{H}(\boldsymbol{x})$, where $\{\boldsymbol{a}_i\}_{i=1}^3$ are lattice translation vectors that span the primitive cell $\Omega := \left\{\boldsymbol{x}\in\mathbb{R}^3 : \boldsymbol{x} = \sum_{i=1}^3 x_i\boldsymbol{a}_i,\ x_i\in[0,1]\right\}$, $\mathbf{k} = (\mathrm{k}_1,\mathrm{k}_2,\mathrm{k}_3)\in\mathbb{R}^3$ is the wave number vector located in the first Brillouin zone.
- **Shifted nabla operator** By introducing the shifted nabla operator $\nabla_{\mathbf{k}} := \nabla + \mathrm{i}\mathbf{k}I$ (I is the identity), the model problem is equivalent to the following system involved with shifted differential operators:

$$\begin{cases} \nabla_{\mathbf{k}}\times(\varepsilon^{-1}\nabla_{\mathbf{k}}\times\boldsymbol{H}) = \omega^2\boldsymbol{H}, & \text{in } \Omega,\\ \nabla_{\mathbf{k}}\cdot\boldsymbol{H} = 0, & \text{in } \Omega,\\ \boldsymbol{H}(\boldsymbol{x}+\boldsymbol{a}_i) = \boldsymbol{H}(\boldsymbol{x}), & i=1,2,3. \end{cases}$$

- **Coordinate change** We introduce the transform $\boldsymbol{x}\mapsto\boldsymbol{H}(\mathbf{A}\boldsymbol{x})$, where $\mathbf{A} = (\boldsymbol{a}_1,\boldsymbol{a}_2,\boldsymbol{a}_3)$, so that the periodicity is along standard orthonormal basis in $\mathbb{R}^3$.
- **Definition of ε** The dielectric function is defined by

$$\varepsilon^{-1}(\boldsymbol{x}) = \begin{cases} \varepsilon_1, & \boldsymbol{x}\in\Omega_1,\\ I_3, & \boldsymbol{x}\in\Omega\setminus\Omega_1, \end{cases}$$

  where $\Omega_1\subset\Omega$ is a subdomain of Ω representing the material area. ε_1 is a constant 3×3 tensor function. In Paper 1, ε_1 is supposed to be a diagonal matrix (or scalar matrix), which corresponds with the `isotropic` media ; In Paper 2, off-diagonal entries of ε_1 occur, which corresponds with `anisotropic`, `pseudochiral` media.
- **Mimetic finite difference** The magnetic field $\boldsymbol{H}$ is approximated by $3N^3$ face degrees of freedom (DoFs) on an N-uniformly partitioned staggered grid, denoted by $\boldsymbol{H}_h$. The matrix representation of the discrete form is formulated as

$$\begin{cases} (\mathcal{A}\mathcal{M}\mathcal{A}^\dagger)\boldsymbol{H}_h = \omega^2\boldsymbol{H}_h,\\ \mathcal{B}\boldsymbol{H}_h = \mathbf{0}, \end{cases}$$

  where $\mathcal{A}$ stands for the discrete curl operator $\nabla_{\mathbf{k}}\times$ and $\mathcal{B}$ stands for the discrete divergence operator $\nabla_{\mathbf{k}}\cdot$. The conjugate transpose $\mathcal{A}^\dagger$ is derived by the adjoint relation. $\mathcal{A}$ and $\mathcal{B}$ is expressed as

$$\mathcal{A} = \begin{pmatrix} & -\mathcal{D}_3 & \mathcal{D}_2\\ \mathcal{D}_3 & & -\mathcal{D}_1\\ -\mathcal{D}_2 & \mathcal{D}_1 & \end{pmatrix},\quad \mathcal{B} = (\mathcal{D}_1\quad\mathcal{D}_2\quad\mathcal{D}_3),$$

  where $\mathcal{D}_i = \mathcal{D}_{1,i} + ik_i\mathcal{D}_{0,i}$ for $i=1,2,3$ and $\mathcal{D}_{s,1} = I_{N^2}\otimes\mathbf{D}_s,\ \mathcal{D}_{s,2} = I_N\otimes\mathbf{D}_s\otimes I_N,\ \mathcal{D}_{s,3} = \mathbf{D}_s\otimes I_{N^2}$ for $s=0,1$, $\otimes$ is the Kronecker product. Construction of $\mathcal{M}$, the discrete form of ε^{-1}, will be mentioned afterwards.
- **Kernel compensation** Based on the consistent condition $\mathcal{B}\mathcal{A} = \mathbf{0}$, the kernel compensation technique is introduced to solve the several smallest positive eigenvalues and corresponding eigenvectors of $\mathcal{A}\mathcal{M}\mathcal{A}^\dagger$ such that the divergence-free condition $\mathcal{B}\boldsymbol{H}_h = \mathbf{0}$ is satisfied. It's formulated by

$$\left(\mathcal{A}\mathcal{M}\mathcal{A}^\dagger + \gamma\mathcal{B}^\dagger\mathcal{B}\right)\boldsymbol{H}_h = \omega^2\boldsymbol{H}_h,$$

  where γ is the penalty coefficient. We prove in Paper 2 that, when γ is chosen

$$\gamma = \begin{cases} 4\pi^2, & \|\mathbf{k}\|_2\geq 1 \text{ or } \mathbf{k}=\mathbf{0},\\ 4\pi^2\|\mathbf{k}\|_2^{-2} & 0<\|\mathbf{k}\|_2<1, \end{cases}$$

the eigenpairs solved from kernel compensation are the desired ones. Moreover, the null space of the kernel compensation system, $\mathcal{H} := \ker\left(\mathcal{A}\mathcal{M}\mathcal{A}^\dagger + \gamma\mathcal{B}^\dagger\mathcal{B}\right) = \ker\mathcal{A}^\dagger \cap \ker\mathcal{B}$, satisfies

$$\mathcal{H} = \begin{cases} \{\mathbf{0}\}, & \mathbf{k} \neq \mathbf{0}, \\ \operatorname{span}\{(\mathbf{1_{N^3}}, \mathbf{0}, \mathbf{0}), (\mathbf{0}, \mathbf{1_{N^3}}, \mathbf{0}), (\mathbf{0}, \mathbf{0}, \mathbf{1}_{N^3})\}, & \mathbf{k} = \mathbf{0}. \end{cases}$$

- **Matrix-free operations** We derive the FFT-diagonalized matrix blocks $\mathcal{K}_\mathcal{A}, \mathcal{K}_\mathcal{B}$ via 3D DFT matrices:

$$\mathcal{F}_3 := \operatorname{diag}(\mathcal{F}, \mathcal{F}, \mathcal{F}), \quad \mathcal{F} := \mathbf{F} \otimes \mathbf{F} \otimes \mathbf{F}, \quad \mathbf{F} := (\frac{1}{\sqrt{N}}\exp(2\pi\mathrm{i}\frac{ij}{N}))_{0\le i,j\le N-1}.$$

There holds

$$\mathcal{A} = \mathcal{F}_3\mathcal{K}_\mathcal{A}\mathcal{F}_3^\dagger, \quad \mathcal{B}^\dagger\mathcal{B} = \mathcal{F}_3\mathcal{K}_\mathcal{B}\mathcal{F}_3^\dagger, \quad \text{where}$$
$$\mathcal{K}_\mathcal{A} = \begin{pmatrix} & -\mathcal{K}_3 & \mathcal{K}_2 \\ \mathcal{K}_3 & & -\mathcal{K}_1 \\ -\mathcal{K}_2 & \mathcal{K}_1 & \end{pmatrix}, \quad \mathcal{K}_\mathcal{B} = \begin{pmatrix} \mathcal{K}_1^\dagger \\ \mathcal{K}_2^\dagger \\ \mathcal{K}_3^\dagger \end{pmatrix} (\mathcal{K}_1 \quad \mathcal{K}_2 \quad \mathcal{K}_3).$$

The blocks $\mathcal{K}_i$ are characterized as

$$\mathcal{D}_i = \mathcal{F}\mathcal{K}_i\mathcal{F}^\dagger, \quad i = 1, 2, 3, \quad \text{where} \quad \begin{cases} \mathcal{K}_1 = I_{N^2} \otimes (\Lambda_1 + \mathrm{i}k_1\Lambda_0), \\ \mathcal{K}_2 = I_N \otimes (\Lambda_1 + \mathrm{i}k_2\Lambda_0) \otimes I_N, \\ \mathcal{K}_3 = (\Lambda_1 + \mathrm{i}k_3\Lambda_0) \otimes I_{N^2}. \end{cases}$$

Thus $\mathcal{A}\mathcal{M}\mathcal{A}^\dagger + \gamma\mathcal{B}^\dagger\mathcal{B} = \mathcal{F}_3(\mathcal{K}_\mathcal{A}\mathcal{F}_3^\dagger\mathcal{M}\mathcal{F}_3\mathcal{K}_\mathcal{A}^\dagger + \gamma\mathcal{K}_\mathcal{B})\mathcal{F}_3^\dagger$, eliminating $\mathcal{F}_3$ on both sides:

$$(\mathcal{K}_\mathcal{A}\mathcal{F}_3^\dagger\mathcal{M}_\varepsilon\mathcal{F}_3\mathcal{K}_\mathcal{A}^\dagger + \gamma\mathcal{K}_\mathcal{B})\boldsymbol{x} = \omega^2\boldsymbol{x}, \quad \boldsymbol{x} = \mathcal{F}_3^\dagger\boldsymbol{H}_h.$$

To construct an effective preconditioner, we consider an approximation by neglecting the middle term $\mathcal{F}_3^\dagger\mathcal{M}\mathcal{F}_3$. This yields the following preconditioning matrix:

$$\mathcal{K}_\mathcal{P} := \mathcal{K}_\mathcal{A}\mathcal{K}_\mathcal{A}^\dagger + \gamma\mathcal{K}_\mathcal{B}.$$

Therefore a complete formulation of operations during each iteration is

$$\begin{cases} \text{Original system:} & (\mathcal{K}_\mathcal{A}\mathcal{F}_3^\dagger\mathcal{M}_\varepsilon\mathcal{F}_3\mathcal{K}_\mathcal{A}^\dagger + \gamma\mathcal{K}_\mathcal{B})\boldsymbol{x} = \omega^2\boldsymbol{x}, \\ \text{Preconditioning:} & \boldsymbol{x} \mapsto \mathcal{K}_\mathcal{P}^{-1}\boldsymbol{x}, \quad \mathcal{K}_\mathcal{P} = \mathcal{K}_\mathcal{A}\mathcal{K}_\mathcal{A}^\dagger + \gamma\mathcal{K}_\mathcal{B}. \end{cases}$$

- **Discretization of ε^{-1}** This part is the major contribution of Paper 2 where we design two novel discretization approaches to ε^{-1} and prove that the resulting matrices $\mathcal{M}$ of both approaches are Hermitian positive definite (HPD). Discrete dielectric matrices $\mathcal{M}$ derived from two methods are denoted by $\mathcal{M}_{\text{trivial}}$, $\mathcal{M}_{\text{crossdof}}$ in case of confusion. The detailed derivation process of $\mathcal{M}_{\text{trivial}}$, $\mathcal{M}_{\text{crossdof}}$ can be found in Section 3 of Paper 2.

Physical Parameters

We consider simple cubic (`SC`), face centered cubic (`FCC`) and body centered (`BCC`) lattices in the experiment. The expression of lattice translation vectors $\{\boldsymbol{a}_i\}_{i=1}^3$ are

$$\begin{cases} \text{SC lattice:} & \boldsymbol{a}_1 = (1,0,0)^\top, \ \boldsymbol{a}_2 = (0,1,0)^\top, \ \boldsymbol{a}_3 = (0,0,1)^\top. \\ \text{FCC lattice:} & \boldsymbol{a}_1 = (0,\frac{1}{2},\frac{1}{2})^\top, \ \boldsymbol{a}_2 = (\frac{1}{2},0,\frac{1}{2})^\top, \ \boldsymbol{a}_3 = (\frac{1}{2},\frac{1}{2},0)^\top. \\ \text{BCC lattice:} & \boldsymbol{a}_1 = (-\frac{1}{2},\frac{1}{2},\frac{1}{2})^\top, \ \boldsymbol{a}_2 = (\frac{1}{2},-\frac{1}{2},\frac{1}{2})^\top, \ \boldsymbol{a}_3 = (\frac{1}{2},\frac{1}{2},-\frac{1}{2})^\top. \end{cases}$$

The wave number vector $\mathbf{k}$ is derived from symmetry points and partition points on their connecting lines in the Brillouin zone. These symmetry points are:

$$\begin{cases} \text{SC:} & \Gamma(0,0,0), L(\pi,0,0), M(\pi,\pi,0), N(\pi,\pi,\pi), \\ \text{FCC:} & X(0,2\pi,0), U\left(\frac{\pi}{2},2\pi,\pi\right), L(\pi,\pi,\pi), \Gamma(0,0,0), W(\pi,2\pi,0), K\left(\frac{3\pi}{2},\frac{3\pi}{2},0\right), \\ \text{BCC:} & H'(2\pi,0,0), \Gamma(0,0,0), P(\pi,\pi,\pi), N(\pi,0,\pi), H(0,2\pi,0). \end{cases}$$

Main Implementation of Paper 2

Core Modules

Module	Description
`discretization.py`	Mimetic finite difference discretization, construction of $\mathcal{K}_{\mathcal{A}}, \mathcal{K}_{\mathcal{B}}, \mathcal{K}_{\mathcal{P}}^{-1}$
`lobpcg.py`	Custom Knyazev's Locally Optimal Block Preconditioned Conjugate Gradient eigensolver.
`_kernels.py`	CUDA kernels for Python, imported by `pcfft.py`
`dielectric.py`	I/O Dielectric tensor manipulation
`pcfft.py`	FFT-based matrix-free operations.

We provide several details of the programming:

- **Storage** $\mathcal{K}_{\mathcal{A}}$ is stored by a $3N^3$ double-precision complex array while $\mathcal{K}_{\mathcal{B}}$ and $\mathcal{K}_{\mathcal{P}}$ are respectively stored by a tuple including a $3N^3$ double-precision float array and a $3N^3$ double-precision complex array. The overall storage of these matrix-free operators sums up to $192N^3$ Bytes. If we choose $N = 100$, $N = 120$ and $N = 150$, then the storage is 183.11 MiB, 316.41 MiB and 617.98 MiB respectively.
- **I/O of dielectric indices** For a given lattice geometry, we record the indices of all DoFs located within the material region Ω_1. These indices are categorized and stored in the `dielectric_examples` directory `edge_dofs/` and `volume_dofs/`. Indices are stored as **integer arrays** in binary (`.bin`) format. The indexing process is vectorized and accelerated by the GPU. For standard N^3 grids, the total indexing and pre-processing time is **less than 1 second**.
- **How _kernels.py improves efficiency** Custom `cupy.ElementwiseKernel` implementations for $\mathcal{K}_{\mathcal{A}}$ and $\mathcal{K}_{\mathcal{B}}, \mathcal{K}_{\mathcal{P}}^{-1}$ operators that deliver matrix-free efficiency **close to native C++ CUDA**.
- **Settings of LOBPCG** Our implementation is based on **Knyazev's seminal LOBPCG algorithm**, enhanced with a **soft-locking** mechanism to prevent convergence stagnation. The module provides a flexible framework supporting:
    - **Problem Types**: Simple eigenproblems ($A\boldsymbol{x} = \lambda\boldsymbol{x}$) and general eigenproblems ($A\boldsymbol{x} = \lambda B\boldsymbol{x}$).
    - **Target Eigenvalues**: Configurable for smallest or largest magnitude.
    - **Locking Strategies**: Supports both with and without soft-locking modes.
    - **Core Routine**: The default robust solver invokes `lobpcg_sep_softlock`, which integrates these features for optimal stability.

Running Examples

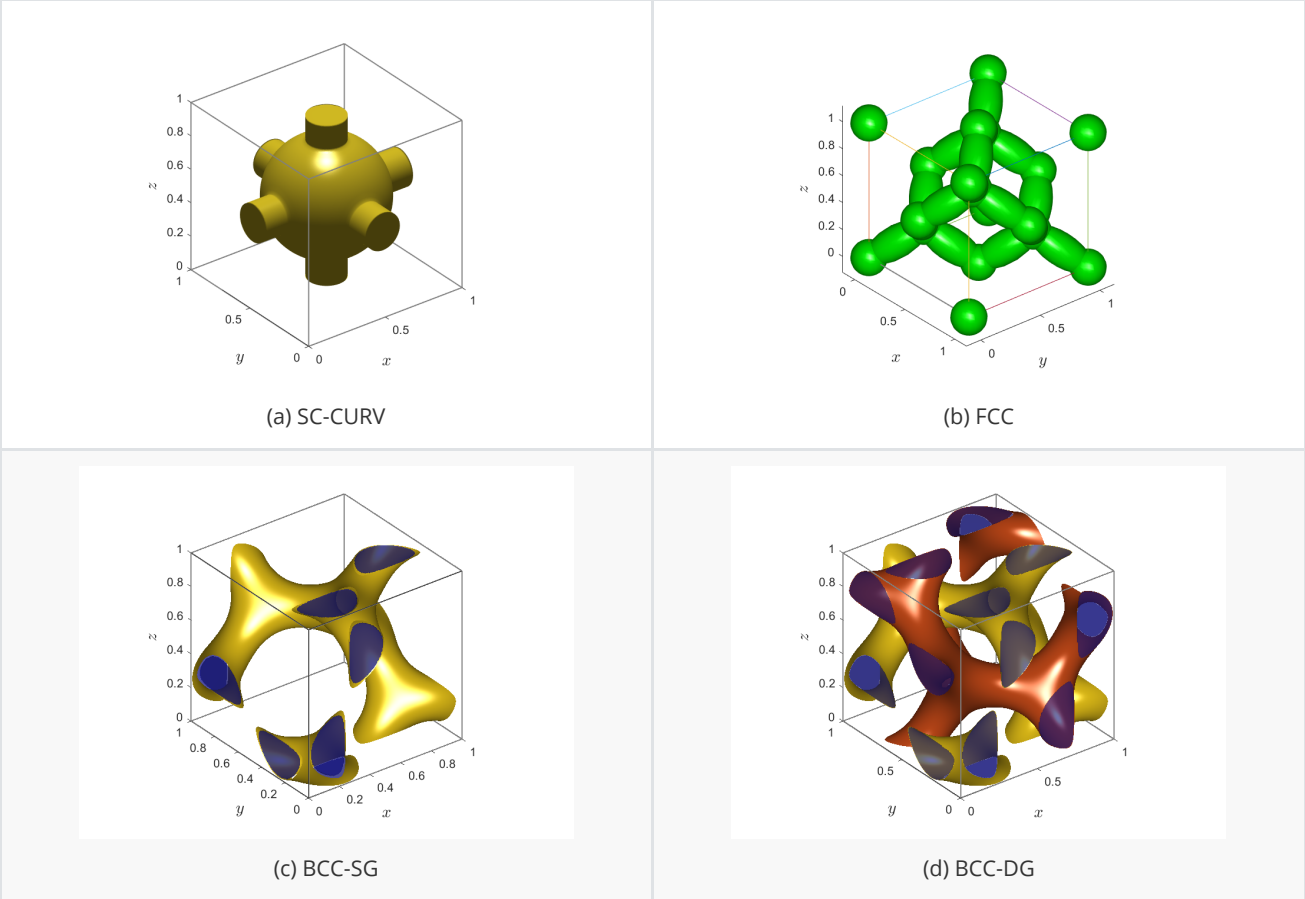
```
cd paper_2

# shell (default: 0, numerical_experiments.py; 2 stands for paper_2_test.py)
./run.sh
./run.sh 0
./run.sh 2

# python
python paper_2_test.py
python numerical_experiments.py
```

Numerical Results

Materials



(a) SC-CURV

(b) FCC

(c) BCC-SG

(d) BCC-DG

Figure: Material structures of four sample lattices in cubic domain.

Bandgap Figures

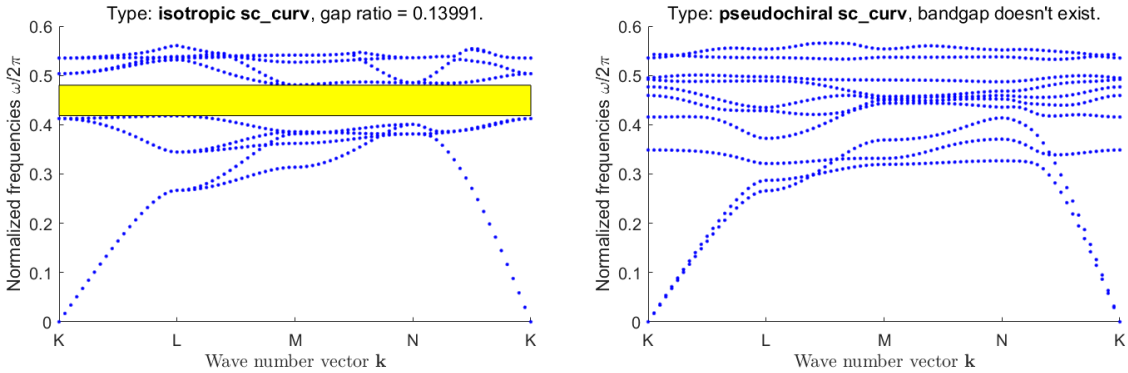


Figure: Band structures of SC lattice with curved material interface. Left: isotropic system; Right: pseudochiral system.

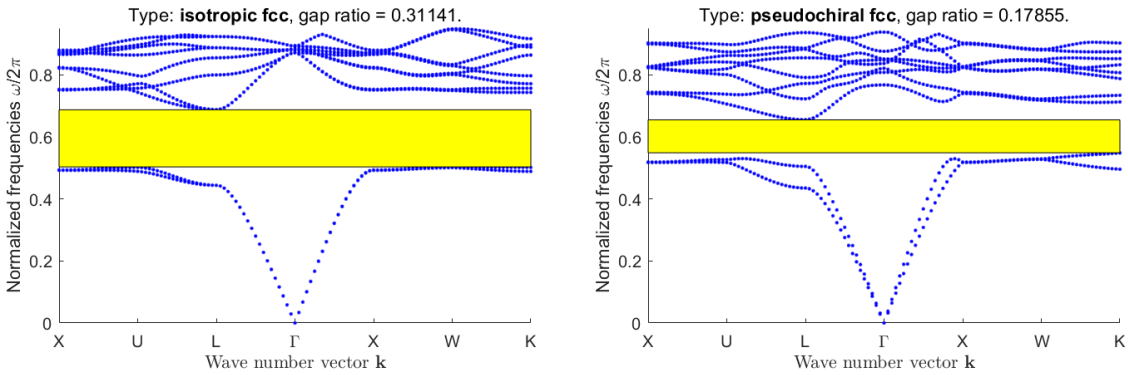


Figure: Band structures of FCC lattice. Left: isotropic system; Right: pseudochiral system.

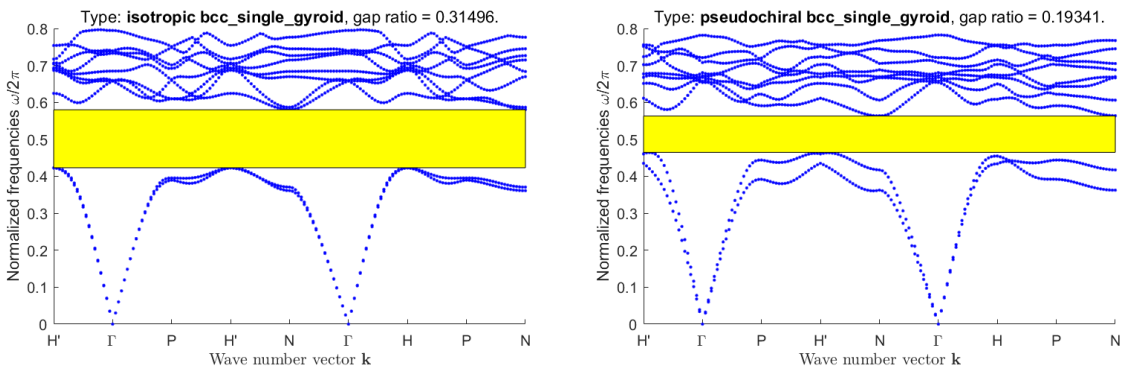


Figure: Band structures of BCC lattice with single gyroid interface. Left: isotropic system; Right: pseudochiral system.

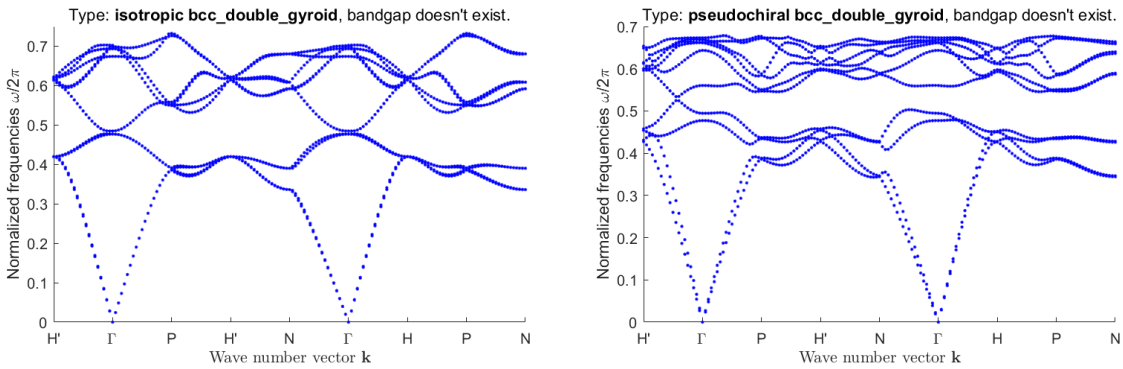


Figure: Band structures of BCC lattice with double gyroid interface. Left: isotropic system; Right: pseudochiral system.

Runtime Tables

The benchmarks presented below are exclusively from **Paper 2** (Python implementation). This version fully covers the numerical experiments from Paper 1 (MATLAB) while delivering a significant performance boost. By leveraging GPU acceleration (CuPy), we achieved an average speedup of **~40x** compared to CPU execution, making the solver highly efficient for large-scale 3D photonic crystal problems.

DoFs	Isotropic SC-CURV				Pseudochiral SC-CURV			
	Steps	GPU time	CPU time	Speed up	Steps	GPU time	CPU time	Speed up
3×100^3	31	10.79	432.71	40.10	50	16.67	666.39	39.98
3×120^3	34	19.85	835.91	42.11	49	28.67	1297.50	45.26
DoFs	**Isotropic FCC**				**Pseudochiral FCC**			
	Steps	**GPU time**	**CPU time**	**Speed up**	**Steps**	**GPU time**	**CPU time**	**Speed up**
3×100^3	45	16.00	623.77	39.01	56	19.59	852.17	43.50
3×120^3	45	27.71	1197.90	43.23	55	34.15	1506.35	44.11
DoFs	**Isotropic BCC-SG**				**Pseudochiral BCC-SG**			
	Steps	**GPU time**	**CPU time**	**Speed up**	**Steps**	**GPU time**	**CPU time**	**Speed up**
3×100^3	57	19.85	835.33	42.08	68	24.14	1020.15	42.26
3×120^3	47	27.96	1295.39	46.33	65	41.08	1816.56	44.22
DoFs	**Isotropic BCC-DG**				**Pseudochiral BCC-DG**			
	Steps	**GPU time**	**CPU time**	**Speed up**	**Steps**	**GPU time**	**CPU time**	**Speed up**
3×100^3	89	26.83	1056.57	39.98	77	25.25	1038.00	41.11

DoFs	Isotropic SC-CURV				Pseudochiral SC-CURV			
	Steps	GPU time	CPU time	Speed up	Steps	GPU time	CPU time	Speed up
3×120^3	86	44.61	1903.39	43.20	76	43.55	1971.94	45.28

Citation

If you use this code, please cite the relevant papers:

```
@article{Jin_2025,
   title={Kernel Compensation Method for Maxwell Eigenproblem in Photonic Crystals With
Mimetic Finite Difference Discretizations},
   volume={41},
   ISSN={1098-2426},
   url={http://dx.doi.org/10.1002/num.23171},
   DOI={10.1002/num.23171},
   number={2},
   journal={Numerical Methods for Partial Differential Equations},
   publisher={Wiley},
   author={Jin, Chenhao and Xia, Yinhua and Xu, Yan},
   year={2025},
   month=feb
}

@misc{jin2025robustgpuacceleratedkernelcompensation,
      title={A Robust GPU-Accelerated Kernel Compensation Solver with Novel
Discretization for Photonic Crystals in Pseudochiral Media},
      author={Chenhao Jin and Hehu Xie},
      year={2025},
      eprint={2511.17107},
      archivePrefix={arXiv},
      primaryClass={math.NA},
      url={https://arxiv.org/abs/2511.17107},
}
```

License

License MIT